

Chuyên đề: Quy Hoạch Động Biên Gãy (Broken Profile DP / Plug DP)

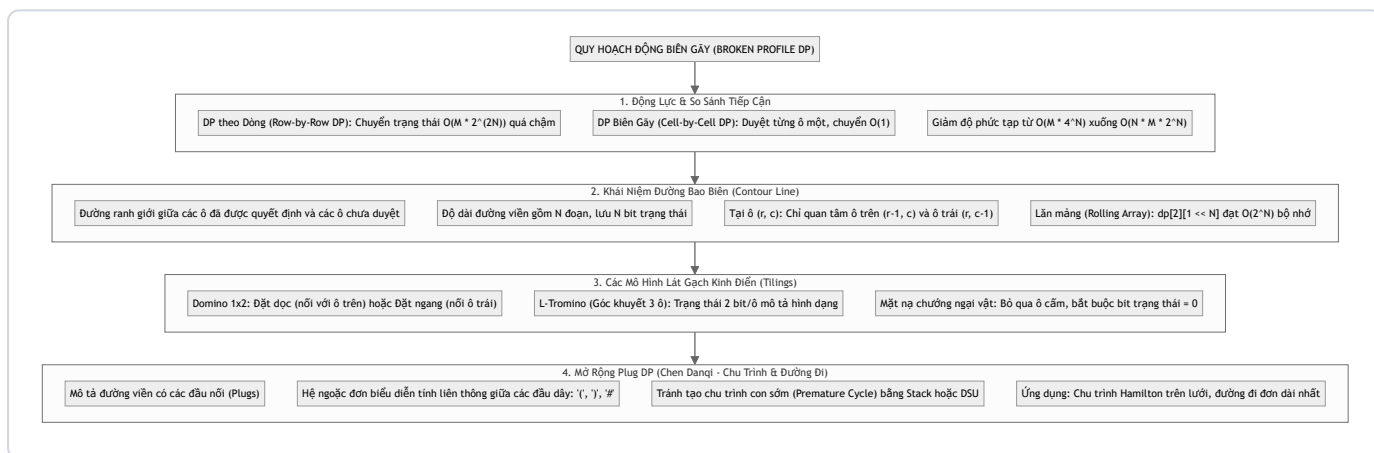
(Contour DP, Domino & Tromino Tilings, Chen Danqi's Plug DP, Hamiltonian Path on Grid, Rolling Bitmask)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 Duc-Minh Vu. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Sơ đồ Tư duy Toàn cảnh (Mindmap)



1. Đặt Vấn Đề: Tại Sao Cần Quy Hoạch Động Biên Gãy?

1.1. Giới hạn của Quy hoạch động theo từng dòng (Row-by-Row DP)

Xét bài toán kinh điển: **Lát gạch Domino 1×2 trên bảng chữ nhật kích thước $N \times M$** (CSES 2181 - Counting Tilings). Cần đếm số cách lát kín bảng bằng các thanh domino 1×2 (hoặc 2×1) không đè lên nhau.

Cách tiếp cận truyền thống là **DP theo từng dòng (Row-by-Row DP)**:

- Gọi $dp[i][mask]$ là số cách lát kín đến hết dòng i , trong đó $mask$ (N bits) biểu thị trạng thái của các ô ở dòng i có bị "chờ" thanh domino dọc xuống dòng $i + 1$ hay không.

- Để chuyển từ dòng i sang dòng $i + 1$, ta phải duyệt qua mọi cặp mặt nạ $(mask, mask')$ và kiểm tra xem có tồn tại cách xếp domino ngang/dọc hợp lệ giữa hai dòng hay không:

$$dp[i + 1][mask'] = \sum_{mask} dp[i][mask] \times \text{valid}(mask, mask')$$

- **Độ phức tạp:** Có M dòng, mỗi bước chuyển giữa 2 dòng cần duyệt $2^N \times 2^N = 4^N$ trạng thái. Tổng độ phức tạp là:

$$O(M \cdot 4^N) \quad \text{hoặc tối ưu DFS thành } O(M \cdot 3^N)$$

- Khi $N = 12$, $4^{12} \approx 1.67 \times 10^7$, tính toán trên $M = 12$ dòng mất hơn 2×10^8 phép tính — rất dễ bị Time Limit Exceeded (TLE).

1.2. Bước đột phá: Quy hoạch động từng ô theo Đường bao biên (Cell-by-Cell DP)

Thay vì chuyển toàn bộ một dòng gồm N ô cùng một lúc, ta chuyển **từng ô đơn lẻ** (r, c) theo thứ tự từ trên xuống dưới, từ trái sang phải (Row-major order):

$$(0, 0) \rightarrow (0, 1) \cdots \rightarrow (0, M - 1) \rightarrow (1, 0) \rightarrow (1, 1) \cdots \rightarrow (N - 1, M - 1)$$

Tại mỗi thời điểm khi đang đứng trước ô (r, c) :

- Các ô đứng trước (r, c) đều **đã được đưa ra quyết định**.
- Các ô đứng sau (r, c) **chưa được xử lý**.
- **Đường bao biên (Broken Profile / Contour Line):** Là đường gấp khúc ngăn cách giữa phần đã xử lý và phần chưa xử lý.

```
Cột:      0    1    2    3    4
Dòng 0: [X] [X] [X] [X] [X]   (Đã phủ kín)
Dòng 1: [X] [X] [?] [ ] [ ]   (Đang xét ô (1, 2))
Dòng 2: [ ] [ ] [ ] [ ] [ ]   (Chưa xét)
```

Đường bao biên gồm đúng N đoạn (chứa N ô ảnh hưởng trực tiếp đến ô hiện tại).

Vì mỗi bước chuyển chỉ quyết định duy nhất cho **1 ô** (r, c) , số lượng nhánh rẽ chỉ là $O(1)$ (tối đa 2 đến 3 nhánh).

Tổng thời gian thực thi:

$$O(N \cdot M \cdot 2^N)$$

Với $N = 12$, $M = 12$: $12 \times 12 \times 2^{12} = 144 \times 4096 \approx 5.89 \times 10^5$ phép tính — chạy trong **chưa đầy 0.01 giây!** Tốc độ nhanh gấp hàng trăm lần so với DP theo dòng!

2. Bản Chất Trạng Thái & Đường Bao Biên (Contour Line)

2.1. Mã hóa Mặt nạ Bit (Bitmask Representation)

Khi chuẩn bị xử lý ô (r, c) , đường bao biên của chúng ta gồm N ô lân cận có nguy cơ "chòì" sang phần chưa xử lý:

- Ô ngay phía trên: $(r - 1, c)$.

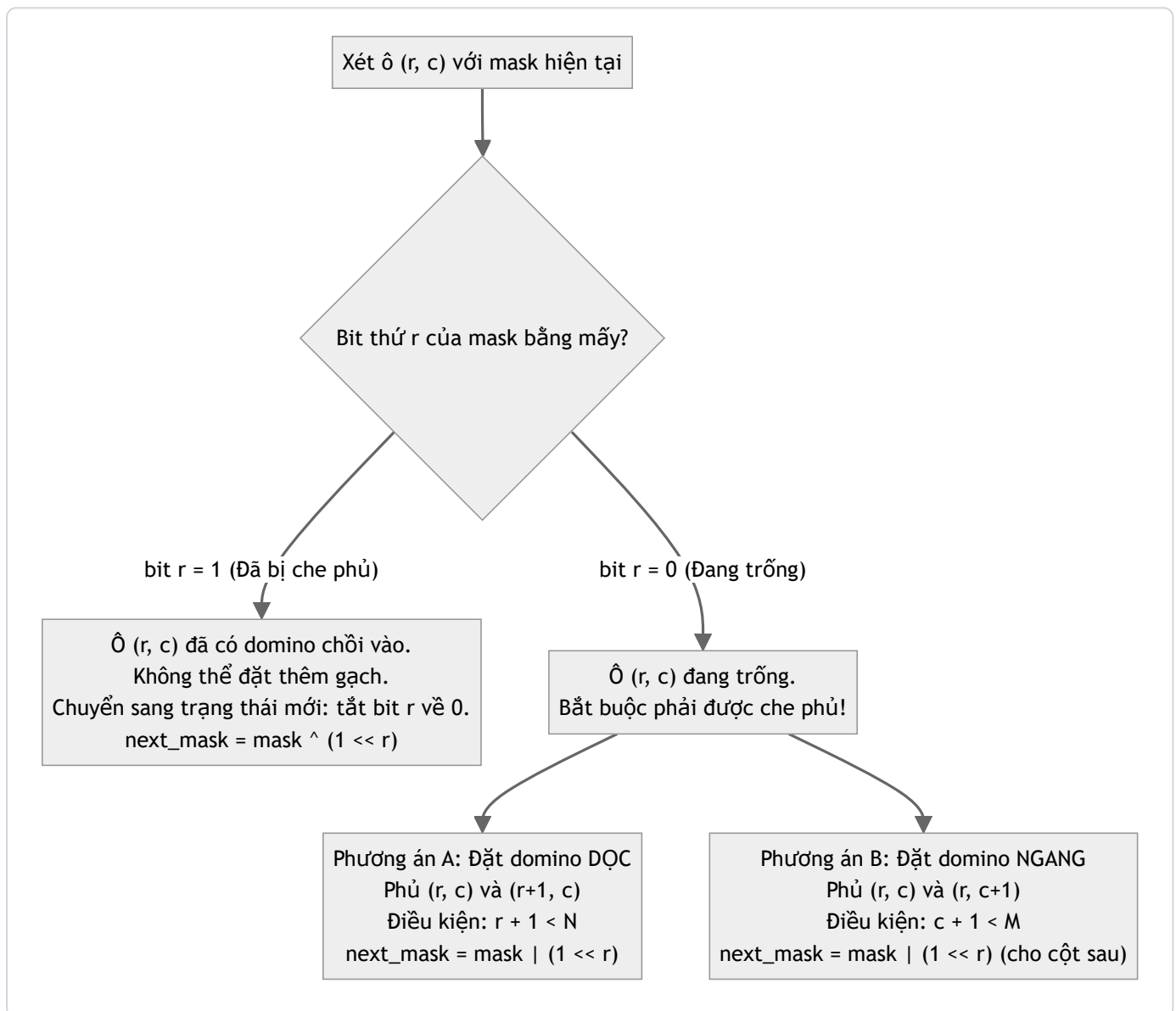
- Các ô ở dòng hiện tại bên trái: $(r, 0 \dots c - 1)$.
- Các ô ở dòng trước bên phải: $(r - 1, c + 1 \dots M - 1)$.

Ta biểu diễn trạng thái đường bao biên bằng một số nguyên *mask* gồm *N* bits (từ bit 0 đến bit *N* - 1):

- Giả sử ta duyệt theo từng cột từ 0 đến *M* - 1, mỗi cột có *N* dòng (hoặc ngược lại, chọn chiều nhỏ hơn làm *N* sao cho $N \leq M$).
- Bit thứ *r* trong *mask*:
 - bit = 0 : Ô tương ứng trên đường biên **đang trống** (chưa được che phủ bởi domino nào).
 - bit = 1 : Ô tương ứng trên đường biên **đã được che phủ** (bởi một domino dọc từ ô trước chỗi xuống, hoặc domino ngang chỗi sang).

2.2. Quy tắc chuyển trạng thái tại ô (r, c)

Xét ô hiện tại (r, c) với *mask* đang xét. Bit thứ *r* của *mask* chính là trạng thái của ô ngay trước đó che phủ vào vị trí này.



Chi tiết toán học các bước chuyển (Transitions):

1. **Trường hợp bit thứ r đang bật ($mask \& (1 \ll r) \neq 0$):**
2. Ô (r, c) đã được phủ bởi một domino từ trước (ví dụ domino dọc từ ô $(r - 1, c)$ thò xuống).
3. Ta **không được phép** đặt thêm bất kỳ quân cờ nào vào ô này.
4. Khi chuyển sang xét ô tiếp theo, vị trí r này sẽ trở thành ô trên của dòng dưới, trạng thái của nó chuyển thành "không chồi xuống" $\implies$ tắt bit r :

$$next_mask = mask \oplus (1 \ll r)$$

$$dp_{next}[next_mask] += dp_{curr}[mask]$$

5. **Trường hợp bit thứ r đang tắt ($mask \& (1 \ll r) == 0$):**
6. Ô (r, c) đang trống. Để bảng được lát kín hoàn toàn, ô (r, c) **bắt buộc phải được che phủ ngay tại bước này**, vì sau này con trỏ duyệt sẽ tiến lên phía trước và không bao giờ quay lại ô này nữa!
7. Có 2 phương án đặt domino:
 - **Phương án 1 (Đặt domino dọc phủ (r, c) và $(r + 1, c)$):**
 - Điều kiện: $r + 1 < N$ (chưa chạm đáy cột/dòng).
 - Quân cờ sẽ chồi xuống ô $(r + 1, c)$, nghĩa là tại thời điểm xét ô $(r + 1, c)$, bit thứ $r + 1$ sẽ ở trạng thái đã phủ $\implies$ bật bit $r + 1$:

$$next_mask = mask | (1 \ll (r + 1))$$

$$dp_{next}[next_mask] += dp_{curr}[mask]$$

- **Phương án 2 (Đặt domino ngang phủ (r, c) và $(r, c + 1)$):**
- Điều kiện: $c + 1 < M$ (chưa chạm biên phải).
- Quân cờ sẽ chồi sang ô bên phải ở cột tiếp theo, nghĩa là bit thứ r của đường biên mới sẽ ở trạng thái đã phủ $\implies$ bật bit r :

$$next_mask = mask | (1 \ll r)$$

$$dp_{next}[next_mask] += dp_{curr}[mask]$$

2.3. Xử lý khi chuyển hết một cột / một dòng

Khi duyệt hết ô cuối cùng của một cột ($r = N - 1$), ta chuyển sang ô đầu tiên của cột tiếp theo ($(r, c) = (0, c + 1)$):

- Đường bao biên lúc này dịch chuyển sang cột mới.
- Các bit trạng thái giữ nguyên ý nghĩa tương đối, ta tiếp tục vòng lặp cho cột $c + 1$.
- Sau khi duyệt xong ô cuối cùng của bảng $(N - 1, M - 1)$, đáp án bài toán chính là:

$$Ans = dp[mask = 0]$$

(vì toàn bộ bảng đã được lát kín, không còn ô nào bị chừa ra ngoài).

3. Tối Ưu Hóa Bộ Nhớ: Kỹ Thuật Lăn Mảng (Rolling Array)

Mỗi bước chuyển chỉ phụ thuộc vào trạng thái của ô ngay trước đó. Do đó:

- Ta **không cần** lưu toàn bộ bảng $dp[N][M][1 \ll N]$.
- Ta chỉ cần duy trì đúng **2 mảng 1 chiều kích thước 2^N** :
- `dp_curr[1 << N]` : Trạng thái của bước trước.
- `dp_next[1 << N]` : Trạng thái của bước hiện tại.
- Sau mỗi ô, ta gán `dp_curr = dp_next`, và xóa sạch `dp_next` về 0.
- **Bộ nhớ tiêu thụ:** $O(2^N)$ số nguyên — khi $N \leq 16$, chỉ tốn $2^{16} \times 4 \text{ bytes} = 256 \text{ KB}$, hoàn toàn nằm gọn trong bộ nhớ đệm L2 Cache của CPU!

4. Mở Rộng: Lát Gạch Phức Tạp (L-Tromino & Chướng Ngại Vật)

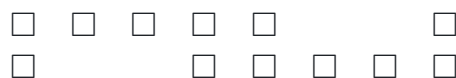
4.1. Bảng có chướng ngại vật (Obstacles)

Nếu ô (r, c) là ô bị cấm (chướng ngại vật, không được đặt gạch lên):

- Nếu bit thứ r đang bật ($mask \& (1 \ll r) \neq 0$): **Vô lý**, vì có quân cờ chèn vào ô cấm $\implies$ Loại bỏ trạng thái này (không cộng tiếp).
- Nếu bit thứ r đang tắt ($mask \& (1 \ll r) == 0$): Ô cấm không bị quân cờ nào đè lên $\implies$ Chuyển tiếp bình thường sang ô sau với $next_mask = mask$, giữ nguyên bit $r = 0$.

4.2. Lát gạch chữ L (L-Tromino / Tromino Tilings)

Thanh Tromino hình chữ L gồm 3 ô vuông (2×2 khoét 1 góc). Có 4 hướng xoay:



Để xử lý L-Tromino:

- Mỗi vị trí trên đường biên có thể cần 2 bits để mô tả trạng thái (hoặc mã hóa hệ cơ số 3 / 4):
- `0` : Chưa bị phủ.
- `1` : Bị phủ bởi phần đầu của Tromino.
- `2` : Đang chờ ghép góc vuông.
- Bằng cách phân tích từng ô (r, c) có thể là góc vuông, đầu mút trên, hay đầu mút dưới của chữ L, ta lập bảng chuyển trạng thái tương tự như Domino nhưng với số lượng nhánh rẽ phong phú hơn.

5. Đỉnh Cao của Quy Hoạch Động Biên Gãy: Plug DP (陈丹琦)

5.1. Giới thiệu Plug DP

Năm 2008, trong kỳ thi chọn đội tuyển Olympic Trung Quốc (CTSC), tuyển thủ **Chen Danqi (Trần Đan Kỳ)** đã công bố bài báo khoa học nổi tiếng: "*Based on the Contours of the Dynamic Programming Method*" (基于连通性状态压缩的动态规划问题).

Phương pháp này mở rộng Broken Profile DP để giải quyết các bài toán **đồ thị trên lưới có ràng buộc liên thông toàn cục**, tiêu biểu là:

- Tìm chu trình Hamilton trên lưới (Hamiltonian Cycle / Circuit).
- Tìm đường đi đơn ngắn nhất / dài nhất nối giữa 2 điểm trên lưới.
- Đếm số cách phủ lưới bằng đúng 1 chu trình kín duy nhất.

5.2. Khái niệm "Phích cắm" (Plug) & Tính Liên Thông

Khi xây dựng một chu trình đi qua các ô của lưới:

- Mỗi ô đi qua sẽ có 2 cạnh nối với các ô kề (vào và ra).
- Các cạnh cắt qua đường bao biên được gọi là các **Phích cắm (Plugs)**.
- Hai phích cắm cắt qua đường biên có thể thuộc cùng một thành phần liên thông của đường đi.

... [X] --- [X] ... <- Các đường dây nối cắt qua biên
1 1

Để đảm bảo kết quả cuối cùng tạo thành **đúng 1 chu trình duy nhất** (không bị phân rã thành nhiều chu trình con rời rạc - Premature Subcycles):

Ta phải lưu vết thông tin: *Phích cắm nào đang nối thông với phích cắm nào qua phần đồ thị đã duyệt.*

Biểu diễn trạng thái bằng Dãy Ngoặc Hợp Lệ (Parenthesis Encoding):

Vì các đường đi trên lưới mặt phẳng không bao giờ cắt chéo nhau, cấu trúc liên thông giữa các cặp phích cắm có tính chất ngoặc lồng nhau hoàn hảo:

- **0**: Không có phích cắm nào cắt qua đoạn biên này.
- **1** (Dấu mở ngoặc **(**): Bắt đầu một đoạn đường đi.
- **2** (Dấu đóng ngoặc **)**): Kết thúc đoạn đường đi tương ứng nối với dấu **(** trước đó.

Mỗi vị trí trên đường biên dùng 2 bits (hệ cơ số 4 hoặc cơ số 3).

Khi đứng trước ô (r, c) , ta xét 2 phích cắm tới từ bên trái (Left Plug L) và từ phía trên (Up Plug U):

1. $L == 0$ và $U == 0$: Ô (r, c) có thể không có đường đi qua, hoặc tạo ra một cặp phích cắm mới $(r, c) \rightarrow$ Đặt một phích cắm xuống dưới **(** và một phích cắm sang phải **)**.

2. **Một trong hai bằng 0** ($L > 0, U == 0$ hoặc $L == 0, U > 0$): Kéo dài đường đi tiếp tục đi thẳng hoặc rẽ nhánh.

3. **Cả hai cùng tồn tại** ($L > 0$ và $U > 0$):

- Nếu $L == ($ và $U ==)$: Hai đầu nối thuộc cùng một đường đi $\implies$ Nối chúng lại sẽ **đóng chu trình**. Chỉ được phép làm điều này ở ô hợp lệ cuối cùng của bảng!

- Nếu $L == ($ và $U ==)$: Dùng stack tìm dấu $)$ ghép cặp với U để đổi nó thành $($.
- Nếu $L ==)$ và $U ==)$: Dùng stack tìm dấu $($ ghép cặp với L để đổi nó thành $)$.

Bằng cách băm trạng thái (Hash Table) chỉ lưu các trạng thái ngoặc hợp lệ (Catalan numbers $C_N \ll 4^N$), Plug DP có thể giải các bài toán chu trình Hamilton trên bảng kích thước lên tới 12×12 trong thời gian 0.2 giây!

6. Bảng So Sánh Các Dạng Quy Hoạch Động Lưới

Tiêu chí	Row-by-Row Bitmask DP	Broken Profile DP (Biên gãy)	Plug DP (Chen Danqi)
Đơn vị chuyển trạng thái	Toàn bộ 1 dòng (N ô)	Từng ô đơn lẻ (r, c)	Từng ô đơn lẻ (r, c)
Số trạng thái	$O(2^N)$	$O(2^N)$	$O(C_N)$ (Số Catalan)
Độ phức tạp thời gian	$O(M \cdot 4^N)$	$O(N \cdot M \cdot 2^N)$	$O(N \cdot M \cdot C_N)$
Bộ nhớ yêu cầu	$O(2^N)$	$O(2^N)$ (Rolling)	$O(C_N)$ (Hash Table)
Bài toán tiêu biểu	Lát gạch đơn giản $N \leq 8$	Lát Domino/Tromino $N \leq 16$	Chu trình Hamilton, đường đi đơn

7. Cạm Bẫy Phổ Biến & Kinh Nghiệm Thi Đấu

- Chọn chiều nhỏ làm N :** Vì độ phức tạp lũy thừa theo N (2^N), bảng $N \times M$ luôn phải được xoay sao cho $N \leq M$. Nếu đề cho $N = 100, M = 10$, hãy xoay bảng để chạy với chiều rộng 10 thay vì 100.
- Trần số với modulo:** Số cách lát bảng tăng rất nhanh theo hàm mũ. Luôn thực hiện phép cộng modulo sau mỗi lần chuyển trạng thái: `dp[next] = (dp[next] + dp[curr]) % MOD`.
- Chuyển dòng bị sót bit chồi:** Khi chuyển từ ô cuối cùng của cột $(N - 1, c)$ sang ô đầu tiên của cột tiếp theo $(0, c + 1)$, không có quân cờ dọc nào được phép chồi qua đáy bảng. Điều này tự nhiên được thỏa mãn nếu ta kiểm tra điều kiện $r + 1 < N$ khi đặt domino dọc.
- Tối ưu hằng số:** Dùng mảng phẳng và con trỏ hoán đổi `swap(dp_curr, dp_next)` thay vì copy mảng. Khởi tạo mảng bằng `memset` hoặc chỉ reset các trạng thái có giá trị khác 0.

8. Tài liệu Tham khảo & Đọc thêm

- CP-Algorithms:**
- [Profile Dynamics \(Broken Profile DP\)](#) — Hướng dẫn chi tiết bài toán lát gạch domino từng ô.
- VNOI Wiki:**
- [Quy hoạch động Bitmask](#) — Nền tảng xử lý mặt nạ bit và bài toán lát bảng.
- CSes Problemset:**
- [CSes 2181 - Counting Tilings](#) — Bài tập chuẩn mực kiểm tra thuật toán Broken Profile DP.

7. **Competitive Programmer's Handbook** — *Antti Laaksonen*:

8. [Chapter 10: Bit Manipulation & DP](#) — Kỹ thuật duyệt mặt nạ và nén trạng thái.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.